Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19/03/2026**	**Enrolment No:92301733054**

Aim: To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:
Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:
Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are sonic of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:
Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to troublinstead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porters Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	**stemmed words**
0	connect	connect
1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

Stopword Removal
Stop words are a set of commonly used words in a language. Examples of stop words in English are "a",

‘‘the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?", you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

```
!pip install nltk spacy

!python -m spacy download en_core_web_sm

import nltk

import spacy

from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize

from nltk.stem import PorterStemmer

from nltk import pos_tag, ne_chunk

nltk.download('punkt')

nltk.download('stopwords')

nltk.download('averaged_perceptron_tagger')

nltk.download('maxent_ne_chunker')

nltk.download('words')

text = """Natural language processing is very interesting.

It helps machines understand human language.

Students are learning NLP in college.

This experiment focuses on preprocessing techniques.

Tokenization splits text into words.
```

```
Stopwords removal reduces noise.

Stemming converts words into root forms.

POS tagging identifies parts of speech.

Chunking groups words into phrases.

Dependency parsing shows relationships between words."""

tokens = word_tokenize(text)

print("Tokenization:\n", tokens)

stop_words = set(stopwords.words('english'))

filtered_tokens = [w for w in tokens if w.lower() not in stop_words]

print("\nAfter Stopword Removal:\n", filtered_tokens)

ps = PorterStemmer()

stemmed_words = [ps.stem(w) for w in filtered_tokens]

print("\nStemmed Words:\n", stemmed_words)

pos_tags = pos_tag(filtered_tokens)

print("\nPOS Tags:\n", pos_tags)

grammar = "NP: {<DT>?<JJ>*<NN>}"

chunk_parser = nltk.RegexpParser(grammar)

chunk_tree = chunk_parser.parse(pos_tags)

print("\nChunking (Noun Phrases):")

print(chunk_tree)

nlp = spacy.load("en_core_web_sm")

doc = nlp(text)

print("\nDependency Parsing:")

for token in doc:
```

```
print(token.text, "->", token.dep_, "->", token.head.text)
```

Results:

```
...
    Dependency Parsing:
    Natural -> amod -> language
    language -> compound -> processing
    processing -> nsubj -> is
    is -> ROOT -> is
    very -> advmod -> interesting
    interesting -> acomp -> is
    . -> punct -> is

     -> dep -> .
    It -> nsubj -> helps
    helps -> ROOT -> helps
    machines -> nsubj -> understand
    understand -> ccomp -> helps
    human -> amod -> language
    language -> dobj -> understand
    . -> punct -> helps

     -> dep -> .
    Students -> nsubj -> learning
    are -> aux -> learning
    learning -> ROOT -> learning
    NLP -> dobj -> learning
    in -> prep -> learning
    college -> pobj -> in
    . -> punct -> learning
```

Observation:

- Tokenization splits the text into smaller units (words), making it easier to process
- Stopword removal reduces unnecessary words and improves efficiency
- Stemming reduces words to their root form, but sometimes produces incomplete or incorrect words
- POS tagging helps identify grammatical roles such as noun, verb, adjective, etc.
- Chunking groups words into meaningful phrases like noun phrases
- Dependency parsing shows relationships between words in a sentence
- Overall, preprocessing improves the quality of text data for NLP task

Post Lab Exercise:

Program code:

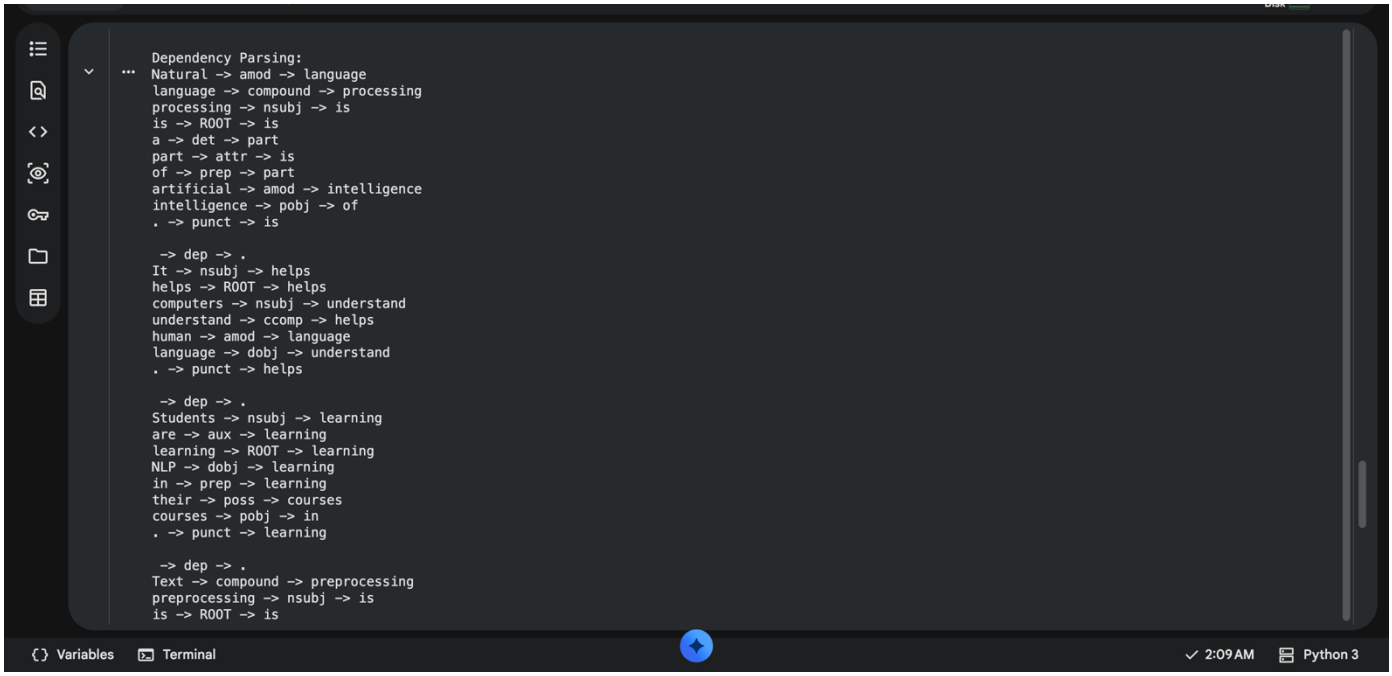
```
!pip install nltk spacy

!python -m spacy download en_core_web_sm

import nltk

import spacy

from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19/03/2026**	**Enrolment No:92301733054**

```
from nltk.stem import PorterStemmer

from nltk import pos_tag, ne_chunk

nltk.download('punkt')

nltk.download('stopwords')

nltk.download('averaged_perceptron_tagger')

nltk.download('maxent_ne_chunker')

nltk.download('words')

text = """Natural language processing is a part of artificial intelligence.

It helps computers understand human language.

Students are learning NLP in their courses.

Text preprocessing is the first step in NLP.

Tokenization breaks text into words.

Stopwords removal improves efficiency.

Stemming reduces words to their base form.

POS tagging identifies parts of speech.

Chunking groups words into phrases.

Dependency parsing shows relationships between words."""

tokens = word_tokenize(text)

print("Tokenization:\n", tokens)

stop_words = set(stopwords.words('english'))

filtered_tokens = [w for w in tokens if w.lower() not in stop_words]

print("\nAfter Stopword Removal:\n", filtered_tokens)

ps = PorterStemmer()

stemmed_words = [ps.stem(w) for w in filtered_tokens]

print("\nStemmed Words:\n", stemmed_words)
```

```
pos_tags = pos_tag(filtered_tokens)

print("\nPOS Tags:\n", pos_tags)

grammar = "NP: {<DT>?<JJ>*<NN>}"

chunk_parser = nltk.RegexpParser(grammar)

chunk_tree = chunk_parser.parse(pos_tags)

print("\nChunking (Noun Phrases):")

print(chunk_tree)

nlp = spacy.load("en_core_web_sm")

doc = nlp(text)

print("\nDependency Parsing:")

for token in doc:

    print(token.text, "->", token.dep_, "->", token.head.text)
```

Output:



```
Dependency Parsing:
Natural -> amod -> language
language -> compound -> processing
processing -> nsubj -> is
is -> ROOT -> is
a -> det -> part
part -> attr -> is
of -> prep -> part
artificial -> amod -> intelligence
intelligence -> pobj -> of
. -> punct -> is

 -> dep -> .
It -> nsubj -> helps
helps -> ROOT -> helps
computers -> nsubj -> understand
understand -> ccomp -> helps
human -> amod -> language
language -> dobj -> understand
. -> punct -> helps

 -> dep -> .
Students -> nsubj -> learning
are -> aux -> learning
learning -> ROOT -> learning
NLP -> dobj -> learning
in -> prep -> learning
their -> poss -> courses
courses -> pobj -> in
. -> punct -> learning

 -> dep -> .
Text -> compound -> preprocessing
preprocessing -> nsubj -> is
is -> ROOT -> is
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study Preprocessing of text (Tokenization. Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	**Date:19/03/2026**	**Enrolment No:92301733054**

Observation:

- Stemming:
  - Removes suffixes using simple rules
  - Faster but may produce incorrect words
  - Example: studies → studi
- Lemmatization:
  - Uses vocabulary and grammar rules
  - Produces meaningful root words
  - Example: studies → study